

Concept de tests du projet WebMall

Informations	
Auteurs	Mathieu Rais, Daniel Baiao
Cours	Module 450
Création du document	26.02.2026
Dernière édition	18.03.2026

Table des matières

1.	Contexte.....	3
2.	Objectifs du concept	3
3.	Objets à tester.....	4
3.1.	Page d'accueil.....	4
3.2.	Jeu promotionnel.....	4
3.3.	Gestion des comptes utilisateurs	4
3.4.	Liste des boutiques	5
3.5.	Plan du centre commercial	5
3.6.	Parking	5
3.7.	Gestion du contenu par les collaborateurs	6
3.8.	Statistiques des visiteurs	6
4.	Types de tests	7
4.1.	Tests unitaires	7
4.2.	Tests d'intégrations.....	8
4.3.	Tests E2E	9
5.	Infrastructure de test	10
5.1.	Environnement de test	10
5.2.	Outils de tests	10
5.3.	Données de test	10
5.4.	Couverture de code	11
6.	Organisation des tests	12
6.1.	Responsabilités.....	12
6.2.	Méthode d'exécution	12
6.3.	Documentation	13
7.	Plan de test.....	14
8.	Classification des défauts.....	15

1. Contexte

Dans le cadre de la création d'un centre commercial, un site web doit être développé afin de présenter les différentes boutiques, fournir des informations utiles aux visiteurs et proposer des fonctionnalités interactives. Le site web devra notamment permettre aux utilisateurs de consulter la liste des magasins présents dans le centre, accéder aux sites officiels des enseignes, visualiser le plan du centre et connaître la disponibilité des places de parking. Afin d'améliorer l'engagement des visiteurs, un jeu sera également disponible sur la page d'accueil. Ce jeu permettra aux utilisateurs inscrits de tenter de gagner des bons d'achat utilisables dans les différentes boutiques du centre commercial. Par ailleurs, le site devra permettre aux collaborateurs du centre de mettre à jour facilement le contenu sans connaissances techniques particulières. Le site devra également enregistrer les statistiques de fréquentation afin de permettre l'analyse du nombre de visiteurs sur des périodes journalières, mensuelles et annuelles.

2. Objectifs du concept

L'objectif principal de ce concept de tests est avant toute chose de définir la stratégie et l'organisation de tests qui seront réalisés sur ce site web.

Ces tests permettront de :

- Vérifier que tous les éléments du cahier des charges sont correctement implémentés et fonctionnent entre eux ou individuellement.
- S'assurer du bon fonctionnement du site côté utilisateurs.
- Détecter des erreurs, bugs ou anomalies avant la mise en production.
- Garantir le respect des contraintes imposées.
- Vérifier la stabilité des statistiques des visiteurs ainsi que les informations affichées.

3. Objets à tester

Les objets correspondent aux différentes fonctionnalités et composants à tester sur le site du centre. Notamment :

3.1. Page d'accueil

La page d'accueil devra afficher les informations principales du site et intégrer le jeu permettant aux utilisateurs de gagner des bons d'achat.

Les éléments à tester sont :

- affichage correct de la page
- accessibilité du jeu
- affichage des informations principales du centre commercial

3.2. Jeu promotionnel

Le jeu permet aux utilisateurs de gagner des bons d'achat dans les boutiques du centre commercial.

Les éléments suivants doivent être vérifiés :

- participation uniquement possible pour les utilisateurs connectés
- limitation à une participation par jour
- possibilité d'une deuxième tentative en cas d'échec au premier essai
- limitation à 10 récompenses distribuées par jour
- attribution correcte des bons d'achat

3.3. Gestion des comptes utilisateurs

Les utilisateurs doivent pouvoir créer un compte afin de participer au jeu.

Les éléments à tester sont :

- création d'un compte utilisateur
- connexion et déconnexion
- gestion des sessions utilisateur
- restriction d'accès au jeu sans connexion

3.4. Liste des boutiques

Le site doit afficher la liste complète des boutiques présentes dans le centre commercial.

Les éléments à tester sont :

- affichage de toutes les boutiques
- exactitude des informations affichées
- présence du lien vers le site officiel de chaque magasin
- bon fonctionnement des liens externes

3.5. Plan du centre commercial

Le site doit proposer un plan permettant aux visiteurs de s'orienter dans le centre commercial.

Les éléments à tester sont :

- affichage correct du plan
- accessibilité depuis le site
- compatibilité avec différents appareils (ordinateur, tablette, smartphone)

3.6. Parking

Le site doit afficher le nombre de places disponibles dans les différents parkings.

Les éléments à tester sont :

- affichage du nombre de places disponibles
- mise à jour correcte des informations
- cohérence des données affichées

3.7. Gestion du contenu par les collaborateurs

Les collaborateurs doivent pouvoir modifier facilement le contenu du site.

Les éléments à tester sont :

- modification des informations du site
- ajout ou modification de boutiques
- mise à jour du contenu sans connaissances techniques

3.8. Statistiques des visiteurs

Le site doit enregistrer le nombre de visiteurs afin de produire des statistiques.

Les éléments à tester sont :

- enregistrement des visites
- consultation des statistiques journalières
- consultation des statistiques mensuelles
- consultation des statistiques annuelles

4. Types de tests

Dans le cadre du développement de l'application du centre commercial, trois niveaux de tests seront mis en œuvre : les tests unitaires, les tests d'intégration et les tests end-to-end (E2E). Ces tests ont pour objectif de garantir le bon fonctionnement de l'application, de détecter les anomalies le plus tôt possible et d'assurer la qualité globale du produit.

4.1. Tests unitaires

Les tests unitaires ont pour objectif de vérifier individuellement les plus petites unités fonctionnelles de l'application, par exemple une fonction, une méthode, un composant ou un module métier.

Dans ce projet, les tests unitaires devront couvrir intégralement au moins une fonctionnalité importante de l'application, en incluant :

- les scénarios nominaux
- les scénarios d'exception
- les cas limites

Un minimum de **50 tests unitaires** sera réalisé.

Les tests unitaires porteront par exemple sur :

- la validation des données utilisateur
- les règles de participation au jeu
- la limitation à une participation par jour
- la gestion de la deuxième tentative en cas d'échec
- la limitation à 10 cadeaux maximum par jour
- la logique d'enregistrement des statistiques visiteurs
- les fonctions de calcul ou de filtrage
- les contrôles d'accès aux fonctionnalités réservées

Les tests unitaires utiliseront des **mocks**, **stubs** et **fakes** lorsque cela sera pertinent, afin d'isoler le composant testé des dépendances externes comme la base de données, les API, les services d'authentification ou les horodatages système.

Le niveau de **code coverage** attendu devra être cohérent avec les objectifs définis dans le concept de test. Une couverture élevée sera recherchée sur les composants critiques, notamment ceux liés au jeu, à l'authentification et aux règles métier.

4.2. Tests d'intégrations

Les tests d'intégration ont pour objectif de vérifier le bon fonctionnement de plusieurs composants lorsqu'ils interagissent entre eux.

Ils permettront de s'assurer que les modules de l'application communiquent correctement, par exemple entre :

- le front-end et l'API
- les contrôleurs et la base de données
- le système d'authentification et la logique métier
- le module du jeu et le système de gestion des gains
- le module statistique et le stockage des visites

Un minimum de **5 tests d'intégration** sera réalisé.

Ces tests couvriront partiellement une fonctionnalité de l'application, également avec :

- des scénarios nominaux
- des scénarios d'exception
- des cas limites

Les tests d'intégration utiliseront eux aussi des données de test pertinentes, ainsi que des **fakes**, **stubs** ou **mocks** pour simuler certains services lorsque cela est nécessaire, tout en laissant collaborer plusieurs vraies couches de l'application.

4.3. Tests E2E

Les tests E2E ont pour objectif de vérifier le comportement global de l'application dans des conditions proches de l'utilisation réelle.

Ils permettent de valider des parcours utilisateur complets, en s'assurant que les fonctionnalités principales fonctionnent de bout en bout.

Dans ce projet, il est prévu :

- au minimum **1 test E2E manuel**
- au minimum **1 test E2E automatisé** avec un outil libre

Les scénarios E2E seront choisis selon leur pertinence métier. Ils porteront prioritairement sur les fonctionnalités les plus importantes pour l'utilisateur final, par exemple :

- création d'un compte puis participation au jeu
- connexion puis tentative de rejouer une deuxième fois dans la même journée
- consultation des boutiques et accès au site officiel d'un magasin
- consultation du plan du centre
- consultation des places disponibles dans les parkings

Les tests E2E automatisés devront être documentés dans un README expliquant les prérequis, la configuration et la procédure d'exécution.

5. Infrastructure de test

Afin d'exécuter correctement les différents niveaux de tests, une infrastructure de test adaptée sera mise en place.

5.1. Environnement de test

Les tests seront réalisés dans un environnement distinct de la production afin d'éviter tout impact sur les données réelles et sur les utilisateurs finaux.

Cet environnement comprendra :

- une version de l'application configurée pour les tests
- une base de données de test ou une base simulée
- des comptes de test
- des données spécifiques permettant de reproduire les scénarios de test

5.2. Outils de tests

Les outils de test utilisés dépendront de la stack technique choisie pour l'application. À titre d'exemple, les outils suivants peuvent être utilisés :

Pour les tests unitaires

- framework de test unitaire du projet
- outil de mesure de couverture de code

Pour les tests d'intégration

- même framework de test, avec base de données de test ou environnement simulé
- outils de mock si nécessaire

Pour les tests E2E automatisés

- outil libre comme **Cypress**, **Playwright** ou **Selenium**

5.3. Données de test

Des données de test seront préparées afin de garantir la pertinence des scénarios. Elles incluront notamment :

- utilisateurs valides et invalides
- comptes déjà inscrits
- participations déjà effectuées le jour même
- lots déjà attribués en nombre partiel ou maximal

- boutiques avec et sans lien externe
- statistiques visiteurs simulées
- états différents des parkings

Ces données pourront être injectées sous forme de :

- **mocks** pour simuler des réponses
- **stubs** pour contrôler certains comportements
- **fakes** pour remplacer des services techniques ou une base de données

5.4. Couverture de code

La couverture de code sera mesurée principalement sur les tests unitaires. L'objectif est de couvrir prioritairement les zones critiques de l'application, notamment :

- authentification
- règles métier du jeu
- limitations quotidiennes
- attribution des cadeaux
- collecte des statistiques

Une forte couverture est attendue sur ces composants, car ce sont eux qui présentent le plus grand risque fonctionnel.

6. Organisation des tests

L'organisation des tests définit qui fait quoi, quand et comment.

6.1. Responsabilités

Développeur

Le développeur est responsable de :

- rédiger les tests unitaires
- rédiger les tests d'intégration
- corriger les anomalies détectées
- maintenir la documentation technique des tests
- vérifier la couverture de code

Testeur ou responsable qualité

Selon l'organisation du projet, cette personne est responsable de :

- relire les scénarios de test
- vérifier la cohérence des cas de test avec le cahier des charges
- exécuter ou superviser les tests E2E manuels
- suivre les anomalies identifiées

Chef de projet ou responsable de validation

Cette personne est chargée de :

- valider les critères de réussite
- décider si l'application est prête pour la livraison
- arbitrer en cas de défauts majeurs restants

6.2. Méthode d'exécution

Les tests seront organisés de manière progressive :

1. développement de la fonctionnalité
2. création des tests unitaires associés
3. exécution et correction des anomalies
4. mise en place des tests d'intégration
5. validation des parcours utilisateur avec les tests E2E
6. documentation des résultats

6.3. Documentation

Chaque type de test devra être documenté afin de permettre son exécution par une autre personne.

Une documentation de type **README** devra être fournie pour :

- les tests unitaires
- les tests d'intégration
- les tests E2E automatisés

Cette documentation devra préciser :

- les prérequis
- l'installation éventuelle
- la commande d'exécution
- la localisation des fichiers de test
- les éventuelles variables d'environnement
- la lecture des résultats

7. Plan de test

Le plan de test définit l'ordre dans lequel les tests seront réalisés.

Phase 1 : tests des fonctionnalités principales

- Affichage du site
- Navigation entre les pages
- Affichage des boutiques
- Fonctionnement des liens externes

Phase 2 : tests du système utilisateur

- Création de compte
- Connexion
- Gestion des sessions

Phase 3 : tests du jeu promotionnel

- Participation au jeu
- Limitation des tentatives
- Distribution des récompenses
- Limitation du nombre de gains

Phase 4 : tests des fonctionnalités supplémentaires

- Affichage du plan du centre
- Affichage du nombre de places de parking
- Consultation des statistiques

Phase 5 : tests finaux

- Tests de performance
- Tests de compatibilité
- Validation globale du site

8. Classification des défauts

Les défauts du site seront classifiés selon leur niveau de gravité que voici :

Défaut critique

Un défaut critique empêche l'utilisation du site ou d'une fonctionnalité importante comme :

- Impossibilité de se connecter.
- Jeu inaccessible.
- Site qui ne s'affiche pas.

Défaut majeur

Un défaut majeur affecte une fonctionnalité importante mais n'empêche pas totalement l'utilisation du site.

Exemples :

- Mauvaise attribution des récompenses
- Statistiques incorrectes
- Liens vers les magasins incorrects

Défaut mineur

Un défaut mineur correspond à un problème peu important n'affectant pas les fonctionnalités principales.

Exemples :

- Erreur d'affichage
- Problème de mise en page
- Faute de texte

Défaut esthétique

Un défaut esthétique concerne uniquement l'apparence du site.

Exemples :

- Couleur incorrecte
- Image mal alignée
- Icône incorrecte